Digitale Forensik

Klausurabgabe

Luis Staudt

Aufgabe 1

Aufgabenstellung

Stellen Sie sicher, dass Sie alle Punkte in der Aufgabenstellung berücksichtigt haben (Upload Dateiname, Prüfsummenvergleich (md5) etc. Welche Sleuthkit und xmount Version kommt bei Ihnen zum Einsatz?

Lösung

MD5 Checksumme

Berechnete Checksumme der Images mithilfe von md5sum:

./images/klausur/20260328_apl.E01 064832e41e7a5c6de39cf78673cd7083

./images/klausur/20260328_apl.E02 a84c87d1e70707e15ee55693aacf189c

./images/klausur/20260328_apl.E03 d8ebfe3194db454f7802f5138d7eb396

./images/klausur/20260328_apl.E04 db2a7cde242f3a53c3cfa9a0b2bc8de9

./images/klausur/20260328_apl.E05 d31f769e0d8daebf7b55e4d083d4dab6

./images/klausur/20260328_apl.E06 6ef5a2879cc4e08e0393bd9152f91378

./images/klausur/20260328_apl.E07 8d3cd4639c9b8107fda96f2eec5b1c8f

./images/klausur/20260328_server.E01 bf18a6bdc9e56b1b30334800b254af15

Sleuthkit Version

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic#   mmls -V
The Sleuth Kit ver 4.12.1
```

xmount Version

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# xmount --version
xmount v0.7.6 Copyright (c) 2008-2018 by Gillen Daniel <gillen.dan@pinguin.lu>

  compile timestamp: Apr  1 2024 08:38:03
  gcc version: 13.2.0
  loaded input libraries:
```

```
    - libxmount_input_aff.so supporting "aff"
    - libxmount_input_ewf.so supporting "ewf"
    - libxmount_input_aewf.so supporting "aewf"
    - libxmount_input_raw.so supporting "raw", "dd"
    - libxmount_input_aaff.so supporting "aaff"
  loaded morphing libraries:
    - libxmount_morphing_combine.so supporting "combine"
    - libxmount_morphing_raid.so supporting "raid0"
    - libxmount_morphing_unallocated.so supporting "unallocated"
```

binwalk Version

Zusätzlich habe ich die binwalk Version überprüft, da diese für die Analyse der Images relevant sein könnte:

```
luis@Alpha-17:~/Programming/GitHub/University/DigitalForensic$ pip show binwalk 2>/dev/null |
    grep -i version
Version: 2.3.3
```

Aufgabe 2

Aufgabenstellung

Ermitteln Sie die Informationen zu den Betriebssystemen der beiden Images. (Wenn verfügbar mit Build Release, Service Pack, Installationsdatum etc.)

Lösung

Linux Server Image

Das Linux Server Image ist verschlüsselt:

```
luis@Alpha-17:~/Programming/GitHub/University/DigitalForensic$ fsstat -o 2048 images/20260328_server.E01
Possible encryption detected (High entropy (7.96))
```

Die Infos musste ich mir daher über binwalk besorgen:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# ewfinfo images/20260328_server.E01 | grep -iE 'operating|system|date|media'
        Acquisition date:       Mon Mar 23 15:01:32 2026
        System date:            Mon Mar 23 15:01:32 2026
        Operating system used:  Linux
Media information
        Media type:             fixed disk
        Media size:             4.0 GiB (4298928128 bytes)
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# strings /mnt/ewf_server/ewf1 | grep -oP 'Linux version \S+'
Linux version 6.12.74+deb13+1-amd64
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# strings /mnt/ewf_server/ewf1 | grep -i 'PRETTY_NAME'
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
```

Es handelt sich also um ein Debian GNU/Linux 13 (trixie) mit Linux Kernel Version 6.12.74+deb13+1-amd64.

Windows Image

Bei dem Windows Image handelt es sich um ein Windows 10 Pro installation. Folgende Infos konnten aus der Registry extrahiert werden:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# regripper -p winver -r /tmp/SOFTWARE
Launching winver v.20200525
winver v.20200525
(Software) Get Windows version & build info

```

```
ProductName                Windows 10 Pro
ReleaseID                  2009
BuildLab                   22621.ni_release.220506-1250
BuildLabEx                 22621.1.amd64fre.ni_release.220506-1250
CompositionEditionID       Enterprise
RegisteredOrganization
RegisteredOwner            Windows-Benutzer
InstallDate                2026-02-16 13:44:42Z
InstallTime                2026-02-16 13:44:42Z
```

Die Reg Datei wurde wie folgt extrahiert:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# fls -pr /dev/loop35p3
    | grep 'SOFTWARE$'
r/r 101808-128-4:       Windows/System32/config/SOFTWARE
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# icat /dev/loop35p3
    101808-128-4 > /tmp/SOFTWARE
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# file /tmp/SOFTWARE
/tmp/SOFTWARE: MS Windows registry file, NT/2000 or above
```

Aufgabe 3

Aufgabenstellung

Anlaysieren Sie Registry Einträge bezüglich USB Devices , LAN und oder WLAN Informationen.

Lösung

Zuerst wurde die SYTEM Registry aus dem Image extrahiert:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# fls -pr /dev/loop35p3 | grep 'SYSTEM'
r/r 37802-128-4:        Windows/Prefetch/SYSTEMPROPERTIESPROTECTION.EX-81A2FDE2.pf
r/r 109603-128-4:       Windows/Prefetch/SYSTEMSETTINGS.EXE-084BB8F9.pf
r/r 105387-128-4:       Windows/Prefetch/SYSTEMSETTINGSBROKER.EXE-8BBE2894.pf
r/r 101804-128-4:       Windows/System32/config/SYSTEM
r/r 101802-128-5:       Windows/System32/config/SYSTEM.LOG1
r/r 102064-128-5:       Windows/System32/config/SYSTEM.LOG2
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# icat /dev/loop35p3 101804-128-4 > /tmp/SYSTEM
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# file /tmp/SYSTEM
/tmp/SYSTEM: MS Windows registry file, NT/2000 or above
```

USB Einträge

Im anschluss wurde nach USB Einträgen gesucht:

```
regripper -p usbdevices -r /tmp/SYSTEM
regripper -p usbstor -r /tmp/SYSTEM
regripper -p devclass -r /tmp/SYSTEM
regripper -p mountdev -r /tmp/SYSTEM
```

Es wurden folgende 4 USB Geräte gefunden:

Gerät 1 – SanDisk 3.2Gen1

Hersteller / Modell	SanDisk 3.2Gen1
VID / PID	`VID_0781` / `PID_5581`
Seriennummer	`010116ed7ac862...` (gekürzt)
Laufwerksbuchstabe	`D:`
Erste Installation	2026-03-23 08:15:11Z
Letztes Entfernen	2026-03-23 08:15:54Z

Das Gerät war lediglich ca. 43 Sekunden angeschlossen (zwischen *Last Arrival* und *Last Removal*), was auf ein schnelles Kopieren oder ein gezieltes kurzes Anschließen (testen, etc) hindeuten kann.

Gerät 2 – CnMemory USB Device

Hersteller / Modell	CnMemory
VID / PID	`VID_090C` / `PID_1000`
Seriennummer	`00040127001068`
Laufwerksbuchstabe	keiner (nur Volume-GUID vorhanden)
Erste Installation	2026-03-23 07:52:45Z
Letztes Entfernen	2026-03-23 08:15:06Z

Dem Gerät wurde kein Laufwerksbuchstabe zugewiesen; es ist lediglich eine Volume-GUID eingetragen (`{46308943-2685-11f1-a521-fa789cc76a14}`).

Gerät 3 – SMI USB DISK (Instanz A)

Hersteller / Modell	SMI USB DISK
VID / PID	`VID_090C` / `PID_2000`
Seriennummer	`03161671000618`
Laufwerksbuchstabe	keiner (nur Volume-GUID vorhanden)
Erste Installation	2026-02-23 13:03:25Z
Letztes Entfernen	nicht erfasst

Gerät 4 – SMI USB DISK (Instanz B)

Hersteller / Modell	SMI USB DISK
VID / PID	`VID_090C` / `PID_1000`
Seriennummer	`AA00000000013226`
Laufwerksbuchstabe	keiner
Erste Installation	2026-02-16 13:42:54Z
Letztes Entfernen	nicht erfasst

LAN / WLAN Einträge

Nun habe ich nach LAN und WLAN Einträgen gesucht:

```
regripper -p nic2 -r /tmp/SYSTEM
regripper -p networklist -r /tmp/SOFTWARE
regripper -p ssid -r /tmp/SOFTWARE
```

ich habe hier die folgenden 3 Adapter gefunden:

Adapter 1 – `{20b88bfa-bfd2-41ba-9779-1ea2a01031d6}`

GUID	`{20b88bfa-bfd2-41ba-9779-1ea2a01031d6}`
LastWrite	2026-02-16 13:42:56Z
DHCP aktiviert	Ja
IP-Adresse	nicht zugewiesen (kein DHCP-Lease)
DNS-Server	nicht konfiguriert
Domain	nicht konfiguriert

Der Adapter ist DHCP-fähig, hat jedoch keine IP-Adresse erhalten. Es liegt kein aktiver Lease vor, was darauf hindeutet, dass zum Zeitpunkt des letzten Schreibens keine Netzwerkverbindung bestand.

Adapter 2 – `{64ac53af-0b3d-11f1-a51a-806e6f6e6963}`

GUID	`{64ac53af-0b3d-11f1-a51a-806e6f6e6963}`
LastWrite	2026-02-16 13:43:02Z
Konfiguration	keine Subkeys vorhanden

Für diesen Adapter sind keinerlei TCP/IP-Parameter hinterlegt. Es handelt sich vermutlich um einen virtuellen oder nicht genutzten Adapter (z. B. ein Loopback- oder VPN-Adapter).

Adapter 3 – `{655f726f-6fea-44e7-a87e-2bea2176bcdb}`

GUID	`{655f726f-6fea-44e7-a87e-2bea2176bcdb}`
LastWrite	2026-03-23 06:53:35Z
DHCP aktiviert	Ja
IP-Adresse	`192.168.2.91`
Subnetzmaske	`255.255.255.0` (/24)
DHCP-Server	`192.168.2.1`
Standard-Gateway	`192.168.2.1`
DNS-Server	`192.168.2.1`
Domain	`fritz.box`
Lease erhalten	2026-03-23 06:53:35Z
T1 (Renewal)	2026-03-28 06:53:35Z
T2 (Rebinding)	2026-04-01 00:53:35Z
Lease läuft ab	2026-04-02 06:53:35Z
Lease-Dauer	864.000 Sekunden (10 Tage)

Dieser Adapter war zum Zeitpunkt der Untersuchung aktiv mit einem Heimnetzwerk verbunden. Die Domain `fritz.box` identifiziert den Router eindeutig als eine **AVM FRITZ!Box**. Gateway und DNS-Server sind identisch (`192.168.2.1`), was dem typischen Heimrouter-Setup entspricht. Die Lease-Dauer von 10 Tagen sowie der Renewal-Zeitpunkt (T1) am 2026-03-28 deuten auf eine stabile, längerfristige Verbindung hin.

Aufgabe 4

Aufgabenstellung

Wieviele Volume Shadow Kopien sind vorhanden? Sind alle lesbar (fls in VSS)?

Lösung

Das Windows NTFS Dateisystem besitzt 3 Shadow Kopien:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# vshadowinfo /dev/
    loop35p3
vshadowinfo 20190323

Volume Shadow Snapshot information:
        Number of stores:       3

Store: 1
        Identifier              : 05564d2f-10a8-11f1-a51f-b0ab8788c71a
        Shadow copy set ID      : 45a4831e-63a8-4841-aec0-a26cdddf3d90
        Creation time           : Feb 23, 2026 13:05:20.966198700 UTC
        Shadow copy ID          : a31e4aea-4dd9-4ec6-aaf0-dfa2c62c59f4
        Volume size             : 118 GiB (127189123072 bytes)
        Attribute flags         : 0x0042000d

Store: 2
        Identifier              : 502f99b2-1158-11f1-a520-b2c693f35e55
        Shadow copy set ID      : 9ca27a02-5288-4608-a7f1-a69fac6e0a58
        Creation time           : Feb 24, 2026 08:21:47.564736000 UTC
        Shadow copy ID          : b04001ee-e2e8-4105-b153-7cbf06e1789b
        Volume size             : 118 GiB (127189123072 bytes)
        Attribute flags         : 0x00020010

Store: 3
        Identifier              : 502f9ad9-1158-11f1-a520-9467f22d4584
        Shadow copy set ID      : bd28bf73-7e88-45e3-bda9-c0e3cc63a901
        Creation time           : Mar 23, 2026 06:54:23.036809000 UTC
        Shadow copy ID          : 26313d01-fbec-452f-973a-284cdad76c21
        Volume size             : 118 GiB (127189123072 bytes)
        Attribute flags         : 0x0042000d
```

Die drei Shadow Kopien wurden mit vshadowmount an /tmp/vshadow/ gemounted und mit fls auf lesbarkeit geprüft:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# fls /tmp/vshadow/vss1
    | head
d/d 1979-144-6: ProgramData
d/d 2124-144-5: Users
r/r 4-128-1:    $AttrDef
r/r 8-128-2:    $BadClus
```

```
r/r 8-128-1:    $BadClus:$Bad
r/r 6-128-4:    $Bitmap
r/r 6-128-5:    $Bitmap:$SRAT
r/r 7-128-1:    $Boot
d/d 11-144-4:   $Extend
r/r 2-128-1:    $LogFile
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# fls /tmp/vshadow/vss2 | head
d/d 1979-144-6: ProgramData
d/d 2124-144-5: Users
r/r 4-128-1:    $AttrDef
r/r 8-128-2:    $BadClus
r/r 8-128-1:    $BadClus:$Bad
r/r 6-128-4:    $Bitmap
r/r 6-128-5:    $Bitmap:$SRAT
r/r 7-128-1:    $Boot
d/d 11-144-4:   $Extend
r/r 2-128-1:    $LogFile
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# fls /tmp/vshadow/vss3 | head
d/d 1979-144-6: ProgramData
d/d 2124-144-5: Users
r/r 4-128-1:    $AttrDef
r/r 8-128-2:    $BadClus
r/r 8-128-1:    $BadClus:$Bad
r/r 6-128-4:    $Bitmap
r/r 6-128-5:    $Bitmap:$SRAT
r/r 7-128-1:    $Boot
d/d 11-144-4:   $Extend
r/r 2-128-1:    $LogFile
```

Aus der Ausgabe lässt sich schließen das alle drei Shadow Kopien lesbar sind, da die Verzeichnisstruktur und die Systemdateien angezeigt werden.

Aufgabe 5

Aufgabenstellung

Analysieren Sie den Unterschied eines PNG Bildes auf dem User Desktop und der Version in der aktuellsten Volume Shadow Kopie. (Gibt es Unterschied? file! Befehl)

Lösung

Das Bild auf dem Desktop wurde mit der Volume Shadow Kopie 3 verglichen. Hierbei handelt es sich um die aktuellste Kopie:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# vshadowinfo /dev/loop35p3 | grep -E 'Store|Creation'
Store: 1
        Creation time           : Feb 23, 2026 13:05:20.966198700 UTC
Store: 2
        Creation time           : Feb 24, 2026 08:21:47.564736000 UTC
Store: 3
        Creation time           : Mar 23, 2026 06:54:23.036809000 UTC
```

Zuerst habe ich das Bild auf dem Desktop gesucht, und aus dem image extrahiert:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# fls -pr /dev/loop35p3 | grep -i 'Users.*Desktop.*\.png$'
r/r 111954-128-5:       Users/baraka/AppData/Local/Microsoft/OneDrive/26.035.0222.0002/Bundle/Assets/common/assets/images/desktop.png
r/r 37795-128-1:        Users/baraka/Desktop/fakeshop_4saleobngymj3qa62k6gyy5cpvzjtf7otanskbyxnsgyql3oqgivd7id.png
r/r 124689-128-4:       Users/baraka/Desktop/Tor Browser/Browser/browser/VisualElements/VisualElements_150.png
r/r 124690-128-4:       Users/baraka/Desktop/Tor Browser/Browser/browser/VisualElements/VisualElements_70.png
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# icat /dev/loop35p3 37795-128-1 > /tmp/desktop_current.png
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# file /tmp/desktop_current.png
/tmp/desktop_current.png: Unicode text, UTF-8 text, with no line terminators
```

In der aktuellen Version des Bildes auf dem Desktop handelt es sich um eine Textdatei und kein PNG Bild. Dies wurde durch ein cat Befehl bestätigt:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# cat /tmp/desktop_current.png | head
Dieses Bild wurde absichtlich zerst rt. Sie finden das originale Bild in der aktuellsten Volume Shadow Kopie.
```

Dann habe ich das Bild aus dem aktuellsten Volume Shadow Kopie extrahiert:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# fls -pr /tmp/vshadow/vss3 | grep -i 'Users.*Desktop.*\.png$'
r/r 111023-128-5:       Users/baraka/AppData/Local/Microsoft/OneDrive/26.032.0217.0003/Bundle/Assets/common/assets/images/desktop.png
r/r 37795-128-1:        Users/baraka/Desktop/fakeshop_4saleobngymj3qa62k6gyy5cpvzjtf7otanskbyxnsgyql3oqgivd7id.png
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# icat /tmp/vshadow/vss3 37795-128-1 > /tmp/desktop_vss.png
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# file /tmp/desktop_vss.png
/tmp/desktop_vss.png: PNG image data, 1362 x 846, 8-bit/color RGB, non-interlaced
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# feh /tmp/desktop_vss.png
libpng error: IDAT: CRC error
```

Extrahierte Bild aus der aktuellsten Volume Shadow Kopie:

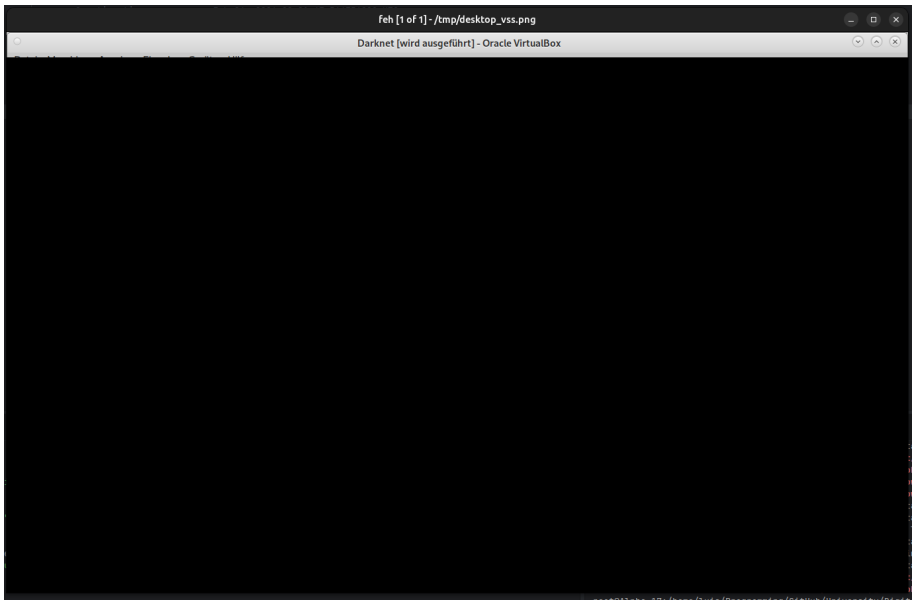

Abbildung 1:

Aufgabe 6

Aufgabenstellung

Welche Kennworthinweise gibt es, um bei einer Virtualisierung des Windows Images das Passwort auszuhebeln?

Lösung

Als erstes wurde die SAM Datei extrahiert, um die Informationen über die Benutzer zu erhalten.

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# fls -pr /dev/loop35p3 | grep 'SAM$'
r/r 100981-128-4:       Windows/System32/config/SAM
d/d 5066-144-1: Windows/System32/LogFiles/SAM
d/d 6223-144-1: Windows/SysWOW64/LogFiles/SAM
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# icat /dev/loop35p3 100981-128-4 > /tmp/SAM
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# file /tmp/SAM
/tmp/SAM: MS Windows registry file, NT/2000 or above
```

Mithilfe von regripper konnte ich die Informationen über die Benutzer extrahieren. Der nutzer "baraka" ist der einzige aktive Nutzer alle anderen Nutzer sind deaktiviert.

```
User Information
------------------------
Username        : baraka [1000]
SID             : S-1-5-21-419137830-851868639-3742987133-1000
Full Name       : baraka
User Comment    :
Account Type    :
Account Created : Mon Feb 16 13:44:37 2026 Z
Name            :
Last Login Date : Mon Mar 23 10:33:52 2026 Z
Pwd Reset Date  : Mon Feb 16 13:50:04 2026 Z
Pwd Fail Date   : Mon Feb 23 11:06:51 2026 Z
Login Count     : 10
  --> Normal user account
  --> Password not required
```

Alle anderen Nuter (Administrator, Gast, DefaultAccount, WDAGUtilityAccount, defaultuser0) sind deaktiviert und wurden aus der Ausgabe oben entfernt (da dies zu viel Platz eingenommen hätten). Da kein Passwort erforderlich ist, könnte man einfach mit einem leeren Passwort versuchen sich einzuloggen.

Aufgabe 7

Aufgabenstellung

Gibt es Spuren im Serverimage bzgl. der Installation des Tor Dienstes? Gibt es einen Webauftritt?

Lösung

Da das Image verschlüsselt ist (siehe Aufgabe 2), erfolgt die Analyse auf dem raw image. Hierfür wurde das Image zuerst mit ewfmount gemountet:

```
mkdir -p /mnt/ewf_server
ewfmount images/20260328_server.E01 /mnt/ewf_server
```

Mithilfe von strings konnte ich die folgenden Onion Domain und Fragmente des Apache Webservers finden:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# strings /mnt/ewf_server/ewf1 | grep -i '\.onion' | sort -u
Homepage: http://2xzrgzkwdqvcfw2ckas4n6iu4gcyk3mvfh6eq5zt5il3ohz2qbtvssyd.onion/
Not resolving .onion address (RFC 7686)
.onion
.onion.
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# strings /mnt/ewf_server/ewf1 | grep -i 'apache\|nginx\|DocumentRoot' | sort -u | head -10
                    $APACHE2CTL $METH > /dev/null 2>&1
                $APACHE2CTL graceful > /dev/null 2>&1
                $APACHE2CTL start
        [ "$APACHE2_MAINTSCRIPT_NAME" ] || APACHE2_MAINTSCRIPT_NAME="${0##*.}"
                                  [ "$APACHE2_MAINTSCRIPT_NAME" = 'postinst' ] || [ "$APACHE2_MAINTSCRIPT_NAME" = 'preinst' ] ; then
  . $APACHE_ENVVARS
      . $apache_reload
    $apache_reload = "  service apache2$dir_suffix $reload\n";
    $apache_reload = "  systemctl $reload $apache_service\n";
    $apache_service .= $service_suffix
```

Die Installation von Tor konnte jedoch nicht direkt nachgewiesen werden:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# strings /mnt/ewf_server/ewf1 | grep -i 'apt.*install.*tor' | head -10
 _ZN3APT11CacheFilter19PackageIsNewInstallclERKN8pkgCache11PkgIteratorE@APTPKG_7.0 1.1~exp4
 _ZN3APT12StateChanges7InstallERKN8pkgCache11VerIteratorE@APTPKG_7.0 1.1~exp15

    _ZN3APT14CacheSetHelper22canNotFindInstalledVerER12pkgCacheFileRKN8pkgCache11PkgIteratorE@APTPKG_7.0 0.8.0

    _ZN3APT25VersionContainerInterface15getInstalledVerER12pkgCacheFileRKN8pkgCache11PkgIteratorERNS_14CacheSe
    .0 0.8.16~exp9
```

```
_ZN3APT11CacheFilter19PackageIsNewInstallclERKN8pkgCache11PkgIteratorE
_ZN3APT14CacheSetHelper22canNotFindInstalledVerER12pkgCacheFileRKN8pkgCache11PkgIteratorE
_ZN3APT25VersionContainerInterface15getInstalledVerER12pkgCacheFileRKN8pkgCache11PkgIteratorERNS_14CacheSetHel

_ZN3APT12StateChanges7InstallERKN8pkgCache11VerIteratorE
virtual bool APT::Internal::Patterns::PackageIsInstalled::operator()(const pkgCache::
    PkgIterator&)
virtual bool APT::Internal::Patterns::PackageIsInstalled::operator()(const pkgCache::
    VerIterator&)
```

Nach der analyse des squashfs Containers aus Aufgabe 8 würde ich sagen der Linux Server definitiv für Darknet Aktivitäten genutzt wurde. Ein Web Server war aber vermutlich nicht installiert, da es keine Spuren von Apache oder Nginx gibt.

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# strings /tmp/swap.bin
    | grep -i '\.onion' | sort -u
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# strings /tmp/swap.bin
    | grep -i 'tor\|hidden_service' | sort -u | head -20
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# strings /tmp/swap.bin
    | grep -i 'apache\|nginx\|www' | sort -u | head -20
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# binwalk /mnt/
    ewf_server/ewf1 2>&1 | grep -i '.onion|tor|hidden_service|apache|nginx|www'
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic#
```

Aufgabe 8

Aufgabenstellung

Ermitteln Sie mit Binwalk den Inhalt eines squashfs Containers am Ende des Server Images.

Lösung

Im ersten Schritt wurden die letzten 50MB des Images extrahiert, da hier der squashfs Container vermutet wird: Auf diesem extrahierten Image wurde dann mit Binwalk der Inhalt des squashfs Containers extrahiert:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# dd if=/mnt/ewf_server/ewf1 of=/tmp/tail.bin bs=1M skip=$(($(stat -c%s /mnt/ewf_server/ewf1)/1048576 - 50))
50+1 records in
50+1 records out
53243904 bytes (53 MB, 51 MiB) copied, 0.0928973 s, 573 MB/s
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# binwalk /tmp/tail.bin

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
49283072      0x2F00000       Squashfs filesystem, little endian, version 4.0, compression: gzip, size: 3960790 bytes, 3722 inodes, blocksize: 131072 bytes, created: 2026-03-23 12:36:31

root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# dd if=/tmp/tail.bin of=/tmp/squashfs.img bs=1 skip=49283072
3960832+0 records in
3960832+0 records out
3960832 bytes (4.0 MB, 3.8 MiB) copied, 5.8737 s, 674 kB/s
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# dd if=/tmp/tail.bin of=/tmp/squashfs.img bs=1 skip=49283072
3960832+0 records in
3960832+0 records out
3960832 bytes (4.0 MB, 3.8 MiB) copied, 5.85739 s, 676 kB/s
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# file /tmp/squashfs.img
/tmp/squashfs.img: Squashfs filesystem, little endian, version 4.0, zlib compressed, 3960790 bytes, 3722 inodes, blocksize: 131072 bytes, created: Mon Mar 23 12:36:31 2026
```

Auszüge aus dem extrahierten squashfs Container:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# unsquashfs -l /tmp/squashfs.img
squashfs-root/.editorconfig
squashfs-root/.git
squashfs-root/4n622tjddwejdqmb4qsd5zyjpwh7f6awnvdhostckei5tui6ohdla3id.onion
squashfs-root/4n622tjddwejdqmb4qsd5zyjpwh7f6awnvdhostckei5tui6ohdla3id.onion/hostname
squashfs-root/4n622tjddwejdqmb4qsd5zyjpwh7f6awnvdhostckei5tui6ohdla3id.onion/hs_ed25519_public_key
```

```
squashfs-root/4n622tjddwejdqmb4qsd5zyjpwh7f6awnvdhostckei5tui6ohdla3id.onion/
    hs_ed25519_secret_key
```

Es handelt sich definitiv um ein git Repository, welches die Konfiguration eines Tor Hidden Services enthält. Es gibt 3316 Dateien mit der Endung .onion:

```
root@Alpha-17:/home/luis/Programming/GitHub/University/DigitalForensic# unsquashfs -l /tmp/
    squashfs.img | grep -i '.onion' | wc -l
3316
```

Es könnte sich um das Git Repository mkp224o handeln. Ein Tool das zur generierung von Tor Hidden Services genutzt wird.